ASSIGNMENT 2 - CO2

Functions and Object-Oriented Concepts in C++

Name	Logesh P
Register Number	192521351
Course Code	DSA 0113
Programme	B.Tech Information Technology, Saveetha School of Engineering (SIMATS), Chennai
Assignment	Assignment 2 - CO2 (Individual, 50 marks, Bloom's level K5 - Evaluate)
Submission Date	31/10/2026
Compiler	g++ 13.3.0 (Ubuntu), C++17
Compile command	g++ -std=c++17 -Wall -Wextra file.cpp -o file (timings: -O0 and -O2)
GitHub Repository	https://github.com/LogeshParamasivan/DSA0113

Contents. Q1 Call by value / reference / pointer (6) | Q2 Return by reference (4) | Q3 Default arguments and overloading (6) | Q4 Class with an array inside (6) | Q5 Static members and arrays of objects (7) | Q6 Friend and inline functions (8) | Q7 Diagnose, redesign and validate (13) | Appendix: README.md and Git commands.

How the evidence was produced. Every program in this report was compiled with `g++ -std=c++17 -Wall -Wextra` and executed; the outputs, compiler messages and timings shown are the real results of those runs. Predictions are written before the corresponding output is shown. All timings were taken on one machine, best of 3 repetitions, so absolute numbers will differ on another computer but the ratios and trends are what is analysed.

Q1 - Call by Value, by Reference and by Pointer (6 Marks)

Problem: Hostel Fee Concession Updater

The concession function looks correct when it prints the fee inside the function, but the stored fee never changes. Reason: the fee was passed **by value**, so the function reduced a private copy. Below, the function is written three ways (value, reference, pointer) plus a fourth way that returns the new fee.

Q1/q1_three_ways.cpp

```
// Q1 - Hostel Fee Concession: by value, by reference, by pointer, by return
#include <iostream>
#include <string>

// 1. by value: works on a COPY, caller's fee never changes void byValue(long
long fee, int percent) {     fee -= fee * percent / 100;     std::cout << "
inside byValue   : " << fee << '\n';   // looks correct here }

// 2. by reference: 'fee' is another name for the caller's
variable void byReference(long long& fee, int percent) {     fee
-= fee * percent / 100; }

// 3. by pointer: caller passes the address, callee dereferences it void
byPointer(long long* fee, int percent) {     if (fee) *fee -= *fee *
percent / 100;   // null check is the extra duty }

// 4. return the new fee (pure function, caller decides what to
store) long long withConcession(long long fee, int percent) {
return fee - fee * percent / 100; }

// const reference: read-only access with no copy (for large objects) struct
Student { char name[64]; char dept[64]; long long fee; int marks[100]; };
long long feeOf(const Student& s) { return s.fee; }

int main() {     const int MERIT = 10, SPORTS = 15,
SINGLE_PARENT = 25;

    long long f1 = 45000, f2 = 45000, f3 = 45000;     std::cout <<
"Prediction: value=45000  ref=40500  pointer=40500\n";
byValue(f1, MERIT);     byReference(f2, MERIT);     byPointer(&f3,
MERIT);
    std::cout << "stored fee after by value    : " << f1 << '\n';
std::cout << "stored fee after by reference: " << f2 << '\n';
std::cout << "stored fee after by pointer  : " << f3 << '\n';

    // Design decision: several concessions on one student (merit + sports + single
parent)     long long inPlace = 45000;     byReference(inPlace, MERIT);
byReference(inPlace, SPORTS);     byReference(inPlace, SINGLE_PARENT);
    long long chained = withConcession(withConcession(withConcession(45000, MERIT),
SPORTS),                                          SINGLE_PARENT);     std::cout << "three
concessions in place : " << inPlace << '\n';     std::cout << "three concessions returned
: " << chained << '\n';

    Student s{"Asha", "IT", 45000, {0}};     std::cout << "sizeof(long long) =
" << sizeof(long long)              << ", sizeof(Student) = " <<
sizeof(Student)               << ", sizeof(const Student&) ~ pointer = " <<
sizeof(void*) << '\n';     std::cout << "feeOf(const ref) = " << feeOf(s) <<
'\n'; }
```

Prediction (before running)

Version	Predicted stored fee after the call (45000, 10% merit)	Reason
By value	45000	only the local copy becomes 40500
By reference	40500	fee is an alias of the caller's variable: 45000 - 45000*10/100 = 40500
Version	Predicted stored fee after the call (45000, 10% merit)	Reason
By pointer	40500	*fee is the caller's variable: same arithmetic

Observed output

```
$ g++ -std=c++17 -Wall -Wextra q1_three_ways.cpp -o q1_three_ways
$ ./q1_three_ways Prediction: value=45000   ref=40500   pointer=40500    inside
byValue    : 40500 stored fee after by value     : 45000 stored fee after by
```

```
reference: 40500 stored fee after by pointer  : 40500 three concessions in place
: 25819 three concessions returned : 25819 sizeof(long long) = 8, sizeof(Student)
= 536, sizeof(const Student&) ~ pointer = 8 feeOf(const ref) = 45000
```

Observed = predicted. The line 'inside byValue : 40500' shows why the bug is confusing - the local copy is correct, but it is destroyed on return and 45000 remains stored.

Inquiry: where is each parameter stored, and why only two change the fee?

Version	Where the parameter lives	Does it change the caller's fee?
By value (long long fee)	A new local variable in the callee's stack frame (8 bytes) initialised with a copy of the argument.	No - it modifies the copy, which dies at return.
By reference (long long& fee)	No new fee object. The reference is another name for the caller's variable (compilers usually pass its address behind the scenes).	Yes - every use of fee is the caller's object.
By pointer (long long* fee)	A local pointer variable (8 bytes) in the callee's frame holding the caller's address; the fee itself stays in the caller.	Yes - *fee reaches the caller's object. Caller must write &fee and callee should check for nullptr.

When to use a const reference parameter: when the function only *reads* a large or costly-to-copy object (for example the 536-byte Student object above, std::string, std::vector). `const Student&` passes only an address (8 bytes), makes no copy, forbids accidental modification, and can also bind to temporaries. For small built-in types such as int or long long, passing by value is just as cheap and simpler.

Design Decision: several concessions on one student

Merit 10%, sports 15% and single parent 25% applied in that order to 45000: 45000 -> 40500 -> 34425 -> 25819 (integer arithmetic truncates 34425*25/100 = 8606.25 to 8606). Both designs printed 25819.

Decision: return the new fee (`long long withConcession(long long fee, int percent)`). Reasons: (1) calls compose - `withConcession(withConcession(fee, 10), 15)` reads like the business rule; (2) the original fee is not destroyed, so the receipt can still show 45000 and the concession applied (an in-place update loses this and cannot be undone); (3) a pure function is trivially testable and has no side effects, so no hidden change to a student record; (4) the caller who really wants in-place can write `fee = withConcession(fee, p)`. In-place (reference) is justified only when the fee lives inside a large Student record that is updated as one unit. Note also that with integer truncation the *order* of concessions changes the result, so the order must be fixed by policy.

Predict the Output

Q1/q1_predict.cpp

```
#include <iostream>

void change(int a, int& b, int*
c) { a += 10; b += 10;

*c += 10;
}

int& bigger(int& p, int& q) { return p > q ? p : q; }

void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }
int main() { int a = 1, b = 2, c = 3;
change(a, b, &c); std::cout << a << ' ' << b
<< ' ' << c << '\n';

bigger(a, c) = 100; std::cout << a
<< ' ' << c << '\n';

show(1); show(1,
2); show(1, 2,
3); std::cout <<
'\n'; }
```

My predicted output (written before running)

Prediction

```
1 12 13
1 100
16 13 6
```

Actual output

```
$ g++ -std=c++17 -Wall -Wextra q1_predict.cpp -o q1_predict
$ ./q1_predict
1 12 13
1 100
16 13 6
```

Difference: none - the prediction was exactly right (note the space after '6' before the newline).

- **change(a, b, &c)**: a is by value, so a stays 1 (the copy became 11). b is a reference, 2 -> 12. c is passed by pointer, *c += 10 makes 3 -> 13. Prints '1 12 13'.
- **bigger(a, c) = 100**: bigger returns an int& to the larger of a (1) and c (13), i.e. a reference to c, so the assignment writes 100 into c. a stays 1. Prints '1 100'.
- **show(1)**: y and z both come from defaults (5 and 10) -> 1+5+10 = 16.
- **show(1, 2)**: only z is defaulted (10) -> 1+2+10 = 13.
- **show(1, 2, 3)**: no default used -> 6.

Complete the Code

Q1/q1_complete.cpp

```
#include <iostream>

void applyByValue(long long fee, int percent) {     fee -=
fee * percent / 100; // changes only the local copy }

void applyConcession(long long& fee, int
percent) {
    // TODO 1: Reduce the caller's fee by percent using integer arithmetic (call
by     // reference)     fee -= fee * percent / 100; }

int main() {     long long
hostelFee = 45000;
applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee <<
'\n'; }
```

Output (matches the expected output)

```
$ g++ -std=c++17 -Wall -Wextra q1_complete.cpp -o
q1_complete $ ./q1_complete after call by value : 45000
after call by reference: 40500
```

Why call by value leaves 45000: applyByValue receives a copy of hostelFee; fee -= ... changes that copy only, and the copy is destroyed when the function returns. The caller's hostelFee is a different object in a different memory location. By reference, 45000*10/100 = 4500, so 45000 - 4500 = 40500.

Alternative Concept and Complexity Analysis

Instead of changing the fee through a reference, return it: `long long applyConcession(long long fee, int percent) { return fee - fee * percent / 100; }` and the caller writes `hostelFee = applyConcession(hostelFee, 10);`. The time is O(1) in all versions (one multiply, one divide, one subtract). Space copied per call (sizeof measured above):

Parameter passing	long long (8 bytes)	Student (536 bytes)
By value	8 bytes copied	536 bytes copied on every call
By reference / pointer	8 bytes (address)	8 bytes (address)
By const reference	8 bytes (address), read-only	8 bytes (address), read-only

Return the new fee (by value)	8 bytes returned (in a register)	returning a Student copies/moves 536 bytes

Conclusion: all are O(1) in time, but copying a large object by value costs O(s) bytes per call (s = object size, a constant), which matters when called millions of times. For long long, by value is as cheap as a reference, so the returning version is the best design for the fee itself.

Q2 - Return by Reference and Its Dangers (4 Marks)

Problem: Examination Marks Register

highest() returns a reference to the top mark so +2 moderation can be applied directly. A second function wrongly returns a reference to a local variable.

Q2/q2_marks.cpp

```
// Q2 - Return by reference: Examination Marks Register
#include <iostream>
#include <vector>
#include <cstdint>

// returns a reference to the highest mark -> caller can modify the array
element int& highest(int marks[], int n) {     int idx = 0;     for (int i =
1; i < n; ++i)         if (marks[i] > marks[idx]) idx = i;     return
marks[idx]; }

// const reference: read only, no copy, cannot be modified by the
caller const int& highestConst(const int marks[], int n) {     int idx
= 0;     for (int i = 1; i < n; ++i)         if (marks[i] >
marks[idx]) idx = i;     return marks[idx]; }

// copy: caller gets an independent value int highestCopy(const int
marks[], int n) { return highestConst(marks, n); }

// alternatives for the Alternative Concept section int highestIndex(const
int marks[], int n) {     int idx = 0;     for (int i = 1; i < n; ++i)
if (marks[i] > marks[idx]) idx = i;     return idx; } int* highestPtr(int
marks[], int n) { return &marks[highestIndex(marks, n)]; }

int main() {     int marks[5] = {62,
88, 75, 91, 54};

    highest(marks, 5) += 2;                 // moderation +2 on the top
mark     std::cout << "after moderation: ";     for (int m : marks)
std::cout << m << ' ';     std::cout << '\n';

    highest(marks, 5) = 100;                // call on the left of
=     std::cout << "after highest()=100: ";     for (int m :
marks) std::cout << m << ' ';     std::cout << '\n';

    int copy = highestCopy(marks, 5);
    copy = 0;                               // does not touch the
array     std::cout << "copy is now " << copy << '\n';     const int&
cr = highestConst(marks, 5);     // cr = 5;  // error: assignment of
read-only reference     std::cout << "copy changed to 0, array top
still " << cr << '\n';

    // Alternatives: index and pointer     marks[highestIndex(marks, 5)] += 2;
*highestPtr(marks, 5) += 2;     std::cout << "via index and pointer: top = " <<
highestCopy(marks, 5) << '\n';

    // Changed condition: vector growth invalidates references
std::vector<int> v = {62, 88, 75, 91, 54};     int& top = v[3];
std::uintptr_t before = reinterpret_cast<std::uintptr_t>(&v[0]);     std::cout <<
"capacity before: " << v.capacity() << '\n';     for (int i = 0; i < 100; ++i)
v.push_back(i);   // forces reallocation     std::uintptr_t after =
reinterpret_cast<std::uintptr_t>(&v[0]);     std::cout << "capacity after : " <<
v.capacity() << '\n';     std::cout << "buffer moved   : " << std::boolalpha <<
(before != after) << '\n';     (void)top;  // using 'top' now would be undefined
behaviour (dangling reference)

    // Safe version after growth: re-obtain, or keep an
index     std::size_t topIndex = 3;     v[topIndex] += 2;
    std::cout << "index stays valid: v[3] = " << v[topIndex] <<
'\n'; }
```

Output

```
$ g++ -std=c++17 -Wall -Wextra q2_marks.cpp -o
q2_marks $ ./q2_marks after moderation: 62 88 75 93
54 after highest()=100: 62 88 75 100 54 copy is now 0
copy changed to 0, array top still 100 via index and
pointer: top = 104 capacity before: 5 capacity after
: 160 buffer moved   : true index stays valid: v[3] =
93
```

What the output shows

- **Reference lets the caller change the array:** `highest(marks,5) += 2` changed 91 to 93 in the array itself, and `highest(marks,5) = 100` works with the call on the left of '=' because an int& return is an lvalue.
- **Return a copy** (highestCopy): copy was set to 0 but the array still holds 100 - the caller gets an independent value; safe but cannot modify the element, and copies cost sizeof(T).
- **Return a const reference** (highestConst): no copy and the caller can read the element in place, but cannot modify it (`cr = 5;` is a compile error). Still dangerous if the object it refers to dies, so the referent must outlive the call.

Returning a reference to a local variable

Q2/q2_dangling.cpp

```
// Q2 - returning a reference to a LOCAL variable (compile only; do not rely on running it)
#include <iostream>

int& badHighest() {     int local = 99;     // dies when the function
returns     return local;       // warning: reference to local
variable returned }

int main() {     int& r = badHighest();   // r is a
dangling reference     std::cout << r << '\n';  //
undefined behaviour }
```

Compiler warning (g++ -std=c++17 -Wall -Wextra)

```
q2_dangling.cpp: In function 'int& badHighest()':
q2_dangling.cpp:7:12: warning: reference to local variable 'local' returned [-Wreturn-local-
addr]     7 |     return local;       // warning: reference to local variable returned       |
^~~~~
q2_dangling.cpp:6:9: note: declared here     6 |     int local
= 99;     // dies when the function returns
      |         ^~~~~
```

Why it is undefined behaviour: `local` has automatic storage; its lifetime ends when badHighest() returns, so r refers to a dead object (a *dangling reference*). Reading it may print garbage, may appear to work, or may crash; the standard gives no guarantee. When I ran this program it terminated with **Segmentation fault (exit code 139)** but another compiler or optimisation level may behave differently, which is exactly why UB is dangerous. Never rely on it.

Changed Condition: the register is a std::vector that may grow

In the output: capacity before = 5, capacity after 100 push_backs = 160, buffer moved = true. When a vector runs out of capacity it allocates a bigger block, moves the elements there and frees the old block. A reference (or pointer or iterator) obtained earlier, such as `int& top = v[3];`, still points into the *freed old block* and becomes dangling; using it is undefined behaviour. Safe alternatives: keep the index and use v[index] each time (shown: v[3] became 93 safely), or re-obtain the reference after any operation that can grow the vector, or reserve() enough capacity first.

Alternative Concept and Complexity Analysis

Returning an index (int) or a pointer (int*) instead of int&:

Calling code

```
marks[highestIndex(marks, 5)] += 2;   // index: caller indexes the array itself
*highestPtr(marks, 5) += 2;               // pointer: caller dereferences
highest(marks, 5) += 2;                   // reference: no extra syntax
```

Observed: after the reference moderation (100) and then the index and pointer moderations, the top mark was 104, so all three modify the same element. **Time:** all three scan the array once, O(n). **Space:** all O(1) extra (one int index plus the returned 4-8 byte value).

Return type	Caller syntax	Safe if array is replaced by a growing vector?
int& (reference)	highest(m) += 2	No - dangling after reallocation if held; fine if used immediately

int* (pointer)	*highestPtr(m) += 2	No - same invalidation, and can also be null/garbage
int (index)	m[highestIndex(m)] += 2	Yes - index stays valid after growth (becomes wrong only if elements are inserted/erased before it)

Justified response: use the reference only for immediate use (modify the element in the same expression); store an *index* when the position must survive growth of the vector.

Q3 - Default Arguments and Function Overloading (6 Marks)

The compiler error (ambiguous call)

Q3/q3_original.cpp

```
// Q3 - the developer's original overload set (does NOT compile)
#include <iostream>

double charge(double weight)                                    { return weight * 50; } double charge(double weight,
bool express)                    { return weight * 50 + (express ? 40 : 0); } double charge(double weight, bool
express = false, bool insured = false)                                                                  { return weight
* 50 + (express ? 40 : 0) + (insured ? 15 :                                                                    0); }

int main() {     std::cout << charge(2.5) << '\n';          // ambiguous: charge(double) vs 3-
arg with defaults     std::cout << charge(2.5, true) << '\n';    // ambiguous: 2-arg vs 3-arg
with defaults }
```

Compiler output

```
q3_original.cpp: In function 'int main()':
q3_original.cpp:11:24: error: call of overloaded 'charge(double)' is ambiguous   11 |    std::cout << charge(2.5)
<< '\n';           // ambiguous: charge(double) vs 3-arg with defaults       |                        ~~~~~~^~~~
q3_original.cpp:4:8:  note:  candidate:  'double  charge(double)'          4  |  double  charge(double  weight)
{ return weight * 50; }       |        ^~~~~~ q3_original.cpp:6:8: note: candidate: 'double charge(double, bool,
bool)'     6 | double charge(double weight, bool express = false, bool insured = false)       |         ^~~~~~
q3_original.cpp:12:24: error: call of overloaded 'charge(double, bool)' is ambiguous    12 |     std::cout <<
charge(2.5, true) << '\n';    // ambiguous: 2-arg vs 3-arg with defaults       |                  ~~~~~~^~~~~~~~~~
q3_original.cpp:5:8: note: candidate: 'double charge(double, bool)'     5 | double charge(double weight, bool
express)                  { return weight * 50 + (express ? 40 : 0); }       |       ^~~~~~ q3_original.cpp:6:8:
note: candidate: 'double charge(double, bool, bool)'     6 | double charge(double weight, bool express = false,
bool insured = false)
      |        ^~~~~~
```

Explanation. `charge(2.5)` matches `charge(double)` exactly and also matches `charge(double, bool = false, bool = false)` exactly (the defaults fill the missing arguments). Both conversion sequences are identical (exact match), so neither is better and the call is ambiguous. Likewise `charge(2.5, true)` matches the two-argument function and the three-argument one with defaults equally well. The default arguments made the first two functions redundant. (`charge(2.5, true, true)` is fine - only one function takes three arguments.)

Redesign: no ambiguity, short common calls

Q3/q3_courier.cpp

```
// Q3 - Courier Charge Calculator: redesigned with defaults + overloading, no ambiguity
#include <iostream>
#include <type_traits>

const double RATE_PER_KG = 50.0, EXPRESS_FEE = 40.0, INSURANCE_FEE = 15.0;

struct Parcel { double kg; bool express; bool insured; };

// A. weight in grams (int)
double charge(int grams, bool express = false, bool insured = false) {     std::cout << "[int grams]
";     return (grams / 1000.0) * RATE_PER_KG + (express ? EXPRESS_FEE : 0) + (insured ? INSURANCE_FEE
: 0); }

// B. weight in kg (double) double charge(double kg, bool express = false, bool insured
= false) {     std::cout << "[double kg]  ";     return kg * RATE_PER_KG + (express ?
EXPRESS_FEE : 0) + (insured ? INSURANCE_FEE : 0); }

// C. a Parcel object double charge(const Parcel& p) {     std::cout << "[Parcel]     ";
return p.kg * RATE_PER_KG + (p.express ? EXPRESS_FEE : 0) + (p.insured ? INSURANCE_FEE : 0);
}
// Alternative design: one function template, no overloads template <typename W> double
chargeT(W w, bool express = false, bool insured = false) {     double kg =
std::is_integral<W>::value ? w / 1000.0 : static_cast<double>(w);     return kg *
RATE_PER_KG + (express ? EXPRESS_FEE : 0) + (insured ? INSURANCE_FEE : 0); }

int main() {      Parcel p{3.0, true, false};     std::cout << "Predictions: 1->int  2->double  3-
>int  4->double  5->double  6->Parcel\n";    std::cout << "1 charge(500)          = " << charge(500)
<< '\n';     std::cout << "2 charge(2.5)            = " << charge(2.5) << '\n';     std::cout << "3
charge(2)              = " << charge(2) << '\n';       // exact match int: 2 grams     std::cout <<
"4 charge(2.0f)           = " << charge(2.0f) << '\n';    // float->double promotion     std::cout
<< "5 charge(1.5, true, true)= " << charge(1.5, true, true) << '\n';     std::cout << "6 charge(p)
= " << charge(p) << '\n';

    std::cout << "template: " << chargeT(500) << ' ' << chargeT(2.5) << ' ' << chargeT(1.5f, true) <<
'\n'; }
```

Prediction before execution

#	Call	Predicted function	Reasoning
1	charge(500)	int grams	int argument: exact match with int overload (int to double would be a conversion)
2	charge(2.5)	double kg	double literal: exact match
3	charge(2)	int grams	2 is an int: exact match beats int-to-double conversion, so 2 means 2 grams
4	charge(2.0f)	double kg	float to double is a *promotion*; float to int is a *conversion*; promotion wins
5	charge(1.5, true, true)	double kg	first argument double: exact; express and insured supplied
6	charge(p)	Parcel	only the Parcel overload takes a Parcel

Observed output

```
$ g++ -std=c++17 -Wall -Wextra q3_courier.cpp -o q3_courier
$ ./q3_courier
Predictions: 1->int  2->double  3->int  4->double  5->double  6->Parcel
1 charge(500)            = [int grams]  25
2 charge(2.5)            = [double kg]  125
3 charge(2)              = [int grams]  0.1
4 charge(2.0f)           = [double kg]  100
5 charge(1.5, true, true)= [double kg]  130 6 charge(p)              = [Parcel]     190 template: 25 125 115
```

Observed = predicted for all six calls. Values: charge(500) = 0.5 kg x 50 = 25; charge(2.5) = 125; charge(2) = 0.002 kg x 50 = 0.1; charge(2.0f) = 100; charge(1.5,true,true) = 75+40+15 = 130; charge(p) = 3x50 + 40 = 190. Note the design consequence of charge(2): a user who meant 2 kg gets 0.1 - the int version means grams. Also, calls such as charge(2L) (long) would be ambiguous because long to int and long to double are both conversions; adding a long overload or using a template would fix that.

Inquiry: how the compiler chooses, and why defaults go from the right

- **Step 1 - candidate set:** all functions with that name visible at the call.
- **Step 2 - viable set:** functions whose parameter count fits (considering defaults) and to which each argument can be converted.
- **Step 3 - rank each argument:** *exact match* (identity, lvalue-to-rvalue, adding const) is better than *promotion* (float to double, short/char/bool to int) which is better than *standard conversion* (int to double, double to int, int to bool, pointer conversions) which is better than *user-defined conversion*.
- **Step 4 - best function:** the one that is at least as good on every argument and strictly better on at least one. If none, the call is **ambiguous** (compile error). Default argument values play no part in ranking.
- **Defaults from the right:** arguments are matched to parameters left to right, and a call cannot skip a middle argument. If a parameter has a default, every parameter after it must too, otherwise there would be a call like f(1, , 3) that cannot be written. So `void f(int a, int b = 2, int c)` is rejected.

Alternative Concept and Complexity Analysis

Alternative A - a single **Parcel** with optional fields: `double charge(const Parcel& p)` where Parcel{kg, express=false, insured=false}. Alternative B - a **function template** (chargeT in the code above, one body for int/float/double, where integers are taken as grams). Observed: chargeT(500) = 25, chargeT(2.5) = 125, chargeT(1.5f, true) = 115.

Aspect	Overloads + default arguments	Parcel struct	Function template
Overload-resolution time	Compile time, zero run-time cost	None needed (one function)	Compile time (deduction), zero run-time cost
Space copied per call	int/double: 4-8 bytes + 1 byte per bool (registers)	const ref: 8 bytes (by value 16 bytes)	same as the overloads

Adding a new option (e.g. fragile)	Add a 4th parameter to every overload; positional bools are easy to mix up	Add one field with default value; existing callers unchanged; callers can name fields	Add a parameter to the one template body; instantiations multiply
Risk	Ambiguity (as in the original), surprising int-means-grams	Caller must build a Parcel; slightly longer calls	Longer error messages, code bloat per type

Run-time cost is the same for all (O(1) arithmetic). The choice is about maintainability: the Parcel design scales best when new options keep arriving.

Q4 - Specifying a Class with an Array Inside It (6 Marks)

Class specification

SavingsAccount keeps account number, balance, an array of the last 10 transactions and a count as **private** data. Constructor and getBalance() are defined **inside** the class; record(), deposit(), withdraw() and miniStatement() are defined **outside** using the scope resolution operator `SavingsAccount::`.

Q4/q4_account.cpp

```
// Q4 - SavingsAccount mini-statement: array of last 10 transactions inside the class
#include <iostream>
#include <string>

struct Transaction { char type; long amount; };   // 'D' deposit, 'W' withdrawal

class SavingsAccount {     static const int HISTORY = 10;        // change to 50 for
the bank's new requirement     int accNo;     long balance;     Transaction
last[HISTORY];            // private array     int count;
// number of valid entries (<= HISTORY)

    void record(char t, long amt);        // private helper, defined outside

public:     SavingsAccount(int no, long opening) : accNo(no), balance(opening), count(0) {}
// inside     long getBalance() const { return balance; }
// inside     bool deposit(long amt);               // defined outside with ::     bool
withdraw(long amt);              // defined outside with ::     void miniStatement() const;
// defined outside with :: };

// shifting design: when full, drop the oldest by moving every element one place
left void SavingsAccount::record(char t, long amt) {     if (count == HISTORY) {
for (int i = 1; i < HISTORY; ++i) last[i - 1] = last[i];   // O(k) shift         -
-count;     }     last[count++] = {t, amt}; }

bool SavingsAccount::deposit(long
amt) {     if (amt <= 0) return
false;     balance += amt;
record('D', amt);     return true; }

bool SavingsAccount::withdraw(long amt)
{
    if (amt <= 0 || amt > balance) return
false;     balance -= amt;     record('W',
amt);     return true; }

void SavingsAccount::miniStatement() const {     std::cout << "Mini-statement for A/c " << accNo << " (last "
<< count << ")\n";     for (int i = 0; i < count; ++i)         std::cout << "  " << (last[i].type == 'D' ?
"Deposit   " : "Withdrawal") << ' ' << last[i].amount << '\n';     std::cout << "  Balance: " << balance <<
'\n'; }

int main() {     SavingsAccount acc(1001, 5000);     for (int i = 1; i <= 12; ++i) {          // 12
transactions: the 11th and 12th force discarding         if (i % 3 == 0) acc.withdraw(100 * i);
else            acc.deposit(100 * i);     }     acc.miniStatement();     std::cout << "withdraw more
than balance accepted? " << std::boolalpha << acc.withdraw(999999) << '\n'; }
```

Output (12 transactions: the 11th and 12th force the oldest two to be dropped)

```
$ g++ -std=c++17 -Wall -Wextra q4_account.cpp -o q4_account
$ ./q4_account Mini-statement for
A/c 1001 (last 10)
  Withdrawal 300
  Deposit    400
  Deposit    500
  Withdrawal 600
  Deposit    700
  Deposit    800
  Withdrawal 900
  Deposit    1000
  Deposit    1100
  Withdrawal 1200
Balance: 6800
withdraw more than balance accepted? false
```

Hand check: opening 5000; deposits 100+200+400+500+700+800+1000+1100 = 4800; withdrawals 300+600+900+1200 = 3000; 5000 + 4800 - 3000 = **6800** = printed balance. The statement starts at 'Withdrawal 300' (transaction 3), so transactions 1 and 2 were discarded - correct for a last-10 statement. An over-large withdrawal is rejected (false) and not recorded.

Inquiry

- **Why the array must be private:** the array and the count form an invariant (count entries valid, oldest first, and every entry matches a real change to the balance). If public, any code could overwrite or erase history, set count to a wrong value (reading garbage or beyond the array) or change the balance without a matching entry - an audit problem for a bank. Private data can be changed only by deposit() and withdraw(), which keep it consistent. It also lets the internal storage (array, circular buffer, deque) change without breaking users.
- **The 11th transaction, shifting:** when count == 10, move entries 1..9 one place left (9 copies, O(k) with k = N-1), then store the new one in the last slot. Cost per transaction once full: O(N) time, O(1) extra space.
- **The 11th transaction, circular index:** keep a start index of the oldest entry; the new transaction overwrites last[start] and start = (start+1) % 10. Cost O(1) time, O(1) extra space (one int). The statement is then printed starting at start and wrapping around (O(N) to print, same as before).

Design Decision: the bank now wants the last 50 transactions

The design that changes least is the one already written in terms of the constant HISTORY - changing `static const int HISTORY = 10;` to 50 is a one-line change and both the shifting code and the circular code (which uses % HISTORY) adapt, because no function contains the literal 10. The cost: each account object grows from 10 x 16 = 160 bytes of history to 800 bytes, and the shifting version now does up to 49 moves per transaction instead of 9. I would therefore keep the fixed array (simple, contiguous, no heap) and, since the shift cost grows with N, switch to the circular index (still just the private helper record() and miniStatement() change; the public interface is untouched). Q7 measures this.

Alternative Concept and Complexity Analysis

std::deque: `if (d.size() == 10) d.pop_front(); d.push_back(t);` - O(1) per transaction, miniStatement just iterates; space grows to about 10 entries plus the deque's block map and allocated blocks (heap, a few hundred bytes minimum), and copying the account copies the deque. **Circular buffer:** same fixed array plus one int start; O(1) per transaction, no heap.

Design	Time to add a transaction	Space	Notes
Shifting fixed array	O(k), k = elements shifted (N-1 when full)	N x 16 bytes + count	Simplest; contiguous and in order
Circular buffer (start index)	O(1)	N x 16 bytes + count + start	Needs modulo arithmetic; printing wraps around
std::deque	O(1) amortised	~N entries + block-map overhead (heap)	Flexible size; allocation cost; not trivially copyable

Measured values for shifting versus circular appear in Q7 (N = 10: 10.1 vs 4.3 ns per add; N = 50: 17.1 vs 4.3 ns).

Q5 - Static Members and Arrays of Objects (7 Marks)

Student-derived class design

Principle: data that belongs to the whole class should exist only once. Applying it to BusTicket:

- **Static:** `issued` (seats issued so far - one value for all tickets) and `CAPACITY` (bus size - one constant), plus the static member function `count()`.
- **Not static (per object):** `seat` and `name` - every ticket has its own.
- **Where a static data member is defined:** declared inside the class (no storage is allocated by a declaration), and defined *exactly once outside* the class in one source file: `int BusTicket::issued = 0;`. (An integral `static const int` like CAPACITY may be given its value inside the class.)
- **Why a static member function cannot use this or non-static members:** it is called without an object (`BusTicket::count()`), so there is no this pointer; seat and name belong to a particular object, so there is nothing for them to refer to. It may use only static members.
- **How an array of objects is created:** `BusTicket trip[4] = {BusTicket("Asha"), ...}` - the constructor runs once per element in index order 0..3. Because the only constructor is *explicit* and takes a

string, there is no default constructor, so `BusTicket trip[4];` would not compile; every element must be initialised.

- **Count in a loop or function:** every construction increments the shared counter; destruction does not decrement it. Objects created inside a loop or function still count after they are destroyed. (A copy made by the copy constructor does not run the string constructor, so it is not counted - and it would duplicate the seat number.)

Predict the Output

Q5/q5_predict.cpp

```
#include <iostream>

class Token {
static int next;
int id;

public: Token() { id = ++next; } int
get() const { return id; } static int
issued() { return next - 100; } };

int Token::next = 100;

int main()

{
Token counter[3];
Token extra;
std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' <<
extra.get() << ' ' << Token::issued() << '\n';
}
```

My predicted output (written before running)

Prediction

```
101 103 104 4
```

Actual output

```
$ g++ -std=c++17 -Wall -Wextra q5_predict.cpp -o q5_predict
$ ./q5_predict
101 103 104 4
```

Difference: none. The static next starts at 100 because `int Token::next = 100;` is initialised before main() starts (static storage, constant initialisation), only once, not per object. The constructor ran **4 times** (3 for the array counter[3] plus 1 for extra): ids 101, 102, 103 for the array, 104 for extra. issued() = next - 100 = 4.

Complete the Code

Q5/q5_bus.cpp

```
#include <iostream>
#include <string>

class BusTicket {     static int issued;                // shared by
all BusTicket objects     static const int CAPACITY = 3;     int seat;
std::string name;

public:     explicit BusTicket(const std::string& n) :
seat(0), name(n)     {
        // TODO 1: If fewer than CAPACITY tickets have been issued, increase issued and
// give this ticket the next seat number; otherwise leave seat as 0 (waiting list)
if (issued < CAPACITY) {             ++issued;             seat = issued;         }
}

    static int count()     {         // TODO 2: Return
the number of seats issued so far         return issued;
}

    void print() const     {         std::cout << name << ": "
<< (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
                  << '\n';
    }
};

int BusTicket::issued = 0;             // definition of the static data member
```

```
int main()
{
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
BusTicket("Chitra"), BusTicket("Dev")};     for (const
BusTicket& t : trip) t.print();     std::cout << "Issued: " <<
BusTicket::count() << '\n'; }
```

Output (matches expected)

```
$ g++ -std=c++17 -Wall -Wextra q5_bus.cpp -o q5_bus
$ ./q5_bus
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

Test Your Design

Array larger than the capacity (6 tickets, capacity 3):

Output

```
$ g++ -std=c++17 -Wall -Wextra q5_array.cpp -o q5_array
$ ./q5_array Array of 6
(capacity 3):
  A: Seat 1
  B: Seat 2
  C: Seat 3
D: Waiting
  E: Waiting
  F: Waiting
Issued: 3
```

Seats 1-3 go to A, B, C; D, E, F wait; issued stays 3. Now tickets created inside a function called twice:

Q5/q5_tests.cpp

```
// Q5 - test: tickets created inside a function called twice, //
and the alternative designs (global counter, TicketOffice object)
#include <iostream>
#include <string>

class BusTicket {     static int
issued;     static const int
CAPACITY = 3;     int seat;
std::string name; public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    { if (issued < CAPACITY) seat = ++issued; }     static int count() { return issued; }     void print() const
{ std::cout << "  " << name << ": " << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting")) << '\n'; }
}; int BusTicket::issued = 0;

void bookGroup(const char* label)
{
    std::cout << label << '\n';     BusTicket a("P1"), b("P2");         // local objects:
destroyed at the end of this function     a.print(); b.print();     std::cout << "  count()
inside = " << BusTicket::count() << '\n'; }

// Alternative: TicketOffice object passed to every ticket (supports two buses) class TicketOffice {
public: int issued = 0; int capacity; explicit TicketOffice(int c) : capacity(c) {} }; class TicketB {
int seat = 0; public:     TicketB(TicketOffice& o) { if (o.issued < o.capacity) seat = ++o.issued; }
int get() const { return seat; } };

int main() {
bookGroup("First call:");
bookGroup("Second call:");
    std::cout << "count after both calls = " << BusTicket::count() << " (objects died, count did not drop)\n\n";

    TicketOffice busA(2), busB(2);     TicketB
a1(busA), a2(busA), a3(busA), b1(busB);
    std::cout << "Two buses with TicketOffice: a1=" << a1.get() << " a2=" <<
a2.get()               << " a3=" << a3.get() << " (waiting)  b1=" << b1.get() <<
'\n';
}
```

Output

```
$ g++ -std=c++17 -Wall -Wextra q5_tests.cpp -o q5_tests
$ ./q5_tests
First call:
  P1: Seat 1   P2:
Seat 2   count()
inside = 2 Second
call:
```

```
  P1: Seat 3   P2:
Waiting   count()
inside = 3
count after both calls = 3 (objects died, count did not drop)

Two buses with TicketOffice: a1=1 a2=2 a3=0 (waiting)  b1=1
```

How the count behaves: in the first call P1 and P2 get seats 1 and 2 (count 2). They are destroyed when the function returns, but the static count stays 2, because it is not tied to any object. In the second call P1 gets seat 3 and P2 goes to the waiting list; the count is 3 afterwards. This is right for 'seats issued', but if a cancelled ticket should free a seat, the destructor would need to decrement the counter.

Alternative Concept and Complexity Analysis

Alternative A - global variable `int seatsIssued;` **Alternative B - separate TicketOffice object** passed to every ticket (code above): observed a1=1, a2=2, a3=0 (waiting) on bus A and b1=1 on bus B, so two buses run independently.

Aspect	Static data member	Global variable	TicketOffice object
Time	O(1)	O(1)	O(1)
Space	one counter	one counter	one counter per office (per bus)
Encapsulation	Private to the class, only BusTicket changes it	None - any code can change it; name clashes	Counter is inside TicketOffice; changed only through its interface
Two buses at once	No - one counter per class	No - one counter	Yes - one office per bus
Cost to caller	None, automatic	None, but fragile	Must pass the office to every constructor

Conclusion: for a single bus the static member is the best fit (encapsulated and automatic); for several buses, TicketOffice is better.

Q6 - Friend Functions and Inline Functions (8 Marks)

Design analysis

- **Why friend:** minutesBetween(a, b) and earlier(a, b) treat the two Time objects *equally*. As a member a.f(b), a would be the special 'this' object and b an ordinary argument, which is asymmetric and reads oddly for symmetric operations. A non-member function is symmetric but cannot see the private h and m; declaring it a friend gives exactly these two functions access without making the data public.
- **Friend vs member:** a member is called on an object (`a.isEarlier(b)`), has a this pointer and is in the class scope. A friend is *not* a member: called as a normal function (`earlier(a, b)`), has no this, is not affected by public/private where it is declared, but *can* read the class's private and protected members. Friendship is not inherited, not mutual and not transitive.
- **Inline:** `inline` is a *request* that the compiler replace the call by the function body to remove call overhead (for tiny functions called millions of times). The compiler may ignore it, and an optimising compiler may inline functions that are not marked inline. It also permits the same definition in several translation units (headers).
- **What is lost if h and m were public:** encapsulation. Anyone could set m = 75 or h = -3 (the class can no longer guarantee a valid time), and if the representation changed (for example, minutes since midnight) every piece of code touching h and m would break.

Complete the Code

Q6/q6_time.cpp

```
#include <iostream>
#include <cstdlib>

class Time {
int h, m;
```

```
public:     Time(int hh, int mm) : h(hh), m(mm) {}
friend int minutesBetween(const Time& a, const Time& b);
friend bool earlier(const Time& a, const Time& b); };

int minutesBetween(const Time& a, const Time& b) {     // TODO 1:
absolute difference in minutes using private h and m     return
std::abs((a.h * 60 + a.m) - (b.h * 60 + b.m)); }

bool earlier(const Time& a, const Time& b) {     //
TODO 2: true if a is earlier in the day than b
return (a.h * 60 + a.m) < (b.h * 60 + b.m); }

int main() {     Time departure(9, 45),
arrival(13, 10);
    std::cout << minutesBetween(departure, arrival) << ' '
<< std::boolalpha << earlier(arrival, departure) << '\n';
}
```

Output (expected: 205 false)

```
$ g++ -std=c++17 -Wall -Wextra q6_time.cpp -o q6_time
$ ./q6_time
205 false
```

Check by hand: 9:45 = 585 min, 13:10 = 790 min; |790 - 585| = 205. earlier(arrival, departure) asks whether 13:10 is before 9:45: false.

Correctness Check

Q6/q6_tests.cpp

```
// Q6 - correctness tests (hand-verified values in the comments)
#include <iostream>
#include <cstdlib>

class Time {
int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}     friend int minutesBetween(const Time& a, const Time&
b);     friend bool earlier(const Time& a, const Time& b); }; inline int minutesBetween(const Time& a,
const Time& b) { return std::abs((a.h*60+a.m) - (b.h*60+b.m)); } inline bool earlier(const Time& a,
const Time& b)      { return (a.h*60+a.m) < (b.h*60+b.m); }

int main() {     Time t1(11, 50), t2(12, 20), same1(8, 30), same2(8, 30), mid(0, 0), end(23, 59);     std::cout <<
std::boolalpha;     // 11:50 -> 12:20 = 30 min; earlier(11:50,12:20)=true; earlier(12:20,11:50)=false     std::cout <<
"noon test   : " << minutesBetween(t1, t2) << ' ' << earlier(t1, t2) << ' ' << earlier(t2, t1) << '\n';     // same
time twice: 0 min, earlier is false both ways     std::cout << "same time   : " << minutesBetween(same1, same2) << ' '
<< earlier(same1, same2) << ' ' << earlier(same2,         same1) << '\n';     // 00:00 -> 23:59 = 1439 min
std::cout << "day extremes: " << minutesBetween(mid, end) << ' ' << earlier(mid, end) << '\n';     // symmetry: |a-b|
== |b-a|     std::cout << "symmetry    : " << (minutesBetween(t1, t2) == minutesBetween(t2, t1)) << '\n'; }
```

Output

```
$ g++ -std=c++17 -Wall -Wextra q6_tests.cpp -o
q6_tests $ ./q6_tests noon test   : 30 true false
same time   : 0 false false day extremes: 1439 true
symmetry    : true
```

Hand verification: 11:50 to 12:20 = 710 to 740 = 30 minutes; 11:50 is earlier than 12:20 (true) and not vice versa (false). Same time twice: 0 minutes and earlier is false both ways (neither is earlier). 00:00 to 23:59 = 1439. Symmetry holds. The calculation works across noon because both times are converted to minutes since midnight (h*60+m).

Growth Investigation: 10,000,000 calls, with and without inline

Q6/q6_bench.cpp

```
// Q6 - Growth Investigation: 10,000,000 calls, friend vs member vs getter,
//      INLINE is set at compile time:  -DINLINE=inline   or   -DINLINE="__attribute__((noinline))"
#include <iostream>
#include <chrono>
#include <cstdlib>
#include <algorithm>

class Time {     int h, m; public:     Time(int
hh = 0, int mm = 0) : h(hh), m(mm) {}
    INLINE int hour() const   { return h; }                  // getters
    INLINE int minute() const { return m; }     INLINE int minutesTo(const Time& b) const { return
std::abs((b.h*60+b.m) - (h*60+m)); }   // member     INLINE bool isEarlier(const Time& b) const {
```

```
return (h*60+m) < (b.h*60+b.m); }     friend int minutesBetween(const Time& a, const Time& b);
friend bool earlier(const Time& a, const Time& b); };
INLINE int minutesBetween(const Time& a, const Time& b) { return std::abs((a.h*60+a.m) - (b.h*60+b.m)); }
INLINE bool earlier(const Time& a, const Time& b)      { return (a.h*60+a.m) < (b.h*60+b.m); }
INLINE int minutesGetter(const Time& a, const Time& b) { return std::abs((a.hour()*60+a.minute()) -
(b.hour()*60+b.minute())); }

template <typename
F> double run(F f) {
    const int N = 10000000;     double best =
1e9;     for (int rep = 0; rep < 3; ++rep) {
long long sink = 0;         auto t0 =
std::chrono::steady_clock::now();         for
(int i = 0; i < N; ++i) sink += f(i);
auto t1 = std::chrono::steady_clock::now();
volatile long long keep = sink; (void)keep;
        best = std::min(best, std::chrono::duration<double, std::milli>(t1 -
t0).count());     }     return best; }

int main()
{
    static Time ts[1024];     for (int i = 0; i < 1024; ++i) ts[i] =
Time(i % 24, (i * 7) % 60);

    std::cout << "friend   : " << run([](int i){ return minutesBetween(ts[i & 1023], ts[(i + 1) & 1023]) + earlier(ts[i
&          1023], ts[(i + 1) & 1023]); }) << " ms\n";     std::cout << "member    : " << run([](int i){ return ts[i &
1023].minutesTo(ts[(i + 1) & 1023]) + ts[i &         1023].isEarlier(ts[(i + 1) & 1023]); }) << " ms\n";     std::cout
<< "getters  : " << run([](int i){ return minutesGetter(ts[i & 1023], ts[(i + 1) & 1023]); }) << " ms\n"; }
```

Built four ways: `-DINLINE=inline` or `-DINLINE="__attribute__((noinline))"`, each at `-O0` and `-O2`. (The attribute is used for the 'without inline' case because at -O2 GCC may inline an unmarked function anyway.) Time for 10,000,000 iterations (best of 3):

Build	friend (minutesBetween + earlier)	member (minutesTo + isEarlier)	public getters (minutes only)
-O0, inline	63.3 ms	63.9 ms	85.2 ms
-O0, noinline	63.5 ms	62.9 ms	94.6 ms
-O2, inline	16.4 ms	16.0 ms	12.4 ms
-O2, noinline	42.4 ms	27.1 ms	64.0 ms

Explanation:

- **At -O0 the inline keyword changes nothing** (63.3 vs 63.5 ms): with optimisation off GCC does not inline, so every call is a real call. This shows inline is only a request.
- **At -O2 inlining helps clearly:** friend 42.4 to 16.4 ms (2.6x), member 27.1 to 16.0 ms (1.7x), getters 64.0 to 12.4 ms (5.2x). The getter design benefits most because it makes four tiny calls per iteration, and after inlining they vanish. Inlined code also lets the compiler optimise across the call (constant folding, vectorisation).
- **Friend and member cost the same** in each build (differences are measurement noise): a friend or a member is just a function with two parameters, and the this pointer is an ordinary hidden parameter.
- **Caveat:** the getters column does only the minutes difference, while friend and member columns also compute earlier(), so it is not exactly equal work; its -O0 values are higher because of the extra getter calls. The friend/member comparison and the inline/noinline comparison are fair.

Deployment Decision

Keep the friend functions. Evidence: after inlining (-O2) friend, member and getter versions run at the same speed (about 12-16 ms for 10 million calls, 1.2-1.6 ns per call), so speed cannot decide. The deciding factor is encapsulation: friends give access to exactly two named functions, getters publish the representation of every Time to every caller, and members would break the natural symmetry of earlier(a, b). I would also keep both functions inline (or define them in the header) and compile with -O2, because without optimisation inline has no effect. If many more operations on Time are added later, a member or operator-based design (operator<) would be preferable to a growing list of friends.

Alternative Concept and Complexity Analysis

Design	Time per call	Encapsulation
Friend functions	O(1); 1.6 ns/call at -O2 inline (16.4 ms / 10M for two calls)	Only two functions see h and m
Member functions (a.minutesTo(b), a.isEarlier(b))	O(1); same after inlining (16.0 ms)	Inside the class; no extra access granted; asymmetric call syntax
Public getters + free functions	O(1); same after inlining but slower un-inlined (64.0 vs 12.4 ms)	Weakest: every caller can read h and m (though not change them)

Q7 - Diagnose, Redesign and Validate (13 Marks)

Program selected: Q4 - SavingsAccount mini-statement (shifting array). It is the program whose cost depends on the requirement that changes: the bank moves from 10 to 50 transactions. Evidence below comes from the Q4 program (the shifting record() function) generalised with the history size N as a template parameter in `Q7/q7_bench.cpp`.

A. Diagnose the Limitation (2 marks)

Where it slows down: record() in Q4 runs `for (i = 1; i < HISTORY; ++i) last[i-1] = last[i];` on every transaction once the account is full, so one add moves N-1 transactions (16 bytes each). Measured (Section E): shifting one add costs 10.1 ns at N = 10, 17.1 ns at N = 50, 61 ns at N = 500 and 1,740 ns at N = 5000, i.e. it grows with the history size even though only one entry is added.

Characteristic of the design that causes it: the array's *physical order is the chronological order* (oldest always in slot 0). Dropping the oldest therefore forces every other element to move. The work is proportional to the size of the history, not to the one transaction being added: add() is O(N), not O(1).

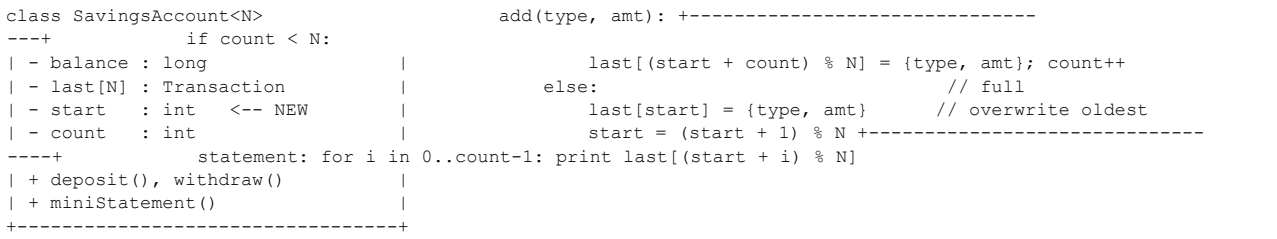
B. Design Objective (2 marks)

Objective: make the cost of adding a transaction independent of the history size (O(1)), measured as: (1) ns per add at N = 50 within 20% of the value at N = 10; (2) at least 3x faster than the shifting version at N = 50; (3) the mini-statement printed must be identical to the original (same checksum on the same transaction sequence); (4) no more than 8 extra bytes per account object.

C. Design the Revised Solution (4 marks)

- **What changes:** the array is treated as a *ring*. A private index `start` marks the oldest entry. Logical entry i lives at `last[(start + i) % N]`. When full, the new transaction *overwrites* the oldest slot and start advances by one (mod N). No element is moved. Only record() and miniStatement() change; the public interface (deposit, withdraw, miniStatement) is unchanged.
- **C++ features used:** the same private array member, an extra private int, the modulo operator, and a class template parameter N (so that history size is a compile-time constant and the array stays a fixed member). No library container is used.
- **Extra memory:** one int (4 bytes). In this layout it sits in padding, so sizeof is identical (176 bytes at N = 10; 80,016 at N = 5000) - measured below.
- **New trade-offs:** (a) printing must wrap around, so miniStatement uses (start + i) % N; (b) more subtle code with off-by-one risk; (c) the data is no longer stored in order, so it cannot be copied or printed with one linear pass; (d) a modulo per access (can be replaced by a compare-and-wrap or a power-of-two mask).

Class diagram and pseudocode



Q7/q7_bench.cpp

```
// Q7 - Original (shifting array) vs Redesigned (circular buffer), history size N is a template parameter
#include <iostream>
#include <chrono>
#include <vector>

struct Transaction { char type; long amount; };

// ---------- ORIGINAL: shift the array left when full --------
-template <int N> class AccountShift {     long balance = 0;
Transaction last[N];     int count = 0; public:
    void add(char t, long amt)     {         if (count == N) {             for (int i = 1; i < N; ++i) last[i - 1] =
last[i];   // O(N)             --count;         }         last[count++] = {t, amt};         balance += (t == 'D' ?
amt : -amt);     }     long checksum() const { long s = balance; for (int i = 0; i < count; ++i) s = s * 31 +
last[i].amount + last[i].type;         return s; } };

// ---------- REDESIGN: circular buffer with a start index ---------template <int N> class
AccountCircular {     long balance = 0;     Transaction last[N];     int start = 0;      // index of
the oldest entry     int count = 0; public:     void add(char t, long amt)     {         if (count <
N) { last[(start + count) % N] = {t, amt}; ++count; }         else           { last[start] = {t,
amt}; start = (start + 1) % N; }   // overwrite oldest, O(1)         balance += (t == 'D' ? amt : -
amt);     }
    long checksum() const { long s = balance; for (int i = 0; i < count; ++i) { const Transaction& x = last[(start +
i) %         N]; s = s * 31 + x.amount + x.type; } return s; } };

template <typename Acc> double timeAdds(int adds, long& cs) {
static Acc acc;      // static: large N would overflow the stack
double best = 1e18;
    for (int rep = 0; rep < 3; ++rep) {         Acc fresh; acc = fresh;         auto t0 =
std::chrono::steady_clock::now();         for (int i = 0; i < adds; ++i) acc.add(i % 3 ? 'D'
: 'W', 100 + (i & 255));         auto t1 = std::chrono::steady_clock::now();         best =
std::min(best, std::chrono::duration<double, std::nano>(t1 - t0).count() / adds);     }
cs = acc.checksum();     return best; }

template <int N> void row(int adds) {     long c1,
c2;     double a = timeAdds<AccountShift<N>>(adds,
c1);     double b =
timeAdds<AccountCircular<N>>(adds, c2);
    std::cout << "N=" << N << "\tshift " << a << " ns/add\tcircular " << b << " ns/add\tspeedup x" << a / b
<< "\tsame statement: " << (c1 == c2 ? "yes" : "NO") << "\tbytes " << sizeof(AccountShift<N>) << '/' <<
sizeof(AccountCircular<N>) << '\n'; }

int main()
{
    const int ADDS = 200000;     row<10>(ADDS); row<50>(ADDS);
row<500>(ADDS); row<5000>(ADDS); }
```

D. Predict Before Implementing (1 mark)

Prediction (made before measuring): the circular version will take a roughly constant few nanoseconds per add at every N (it does one store and one index update). The shifting version will grow with N, approximately linearly. Therefore: speedup at N = 50 of at least 3x, and at N = 5000 of more than 100x. Measure: nanoseconds per add (best of 3 over 200,000 adds, g++ -O2). Both versions must give identical checksums, and the same sizeof.

E. Implement and Validate the Redesign (2 marks)

Raw output (g++ -std=c++17 -O2 q7_bench.cpp)

```
N=10    shift 10.1115 ns/add    circular 4.27102 ns/add speedup x2.36747    same statement: yes bytes 176/176
N=50    shift 17.0526 ns/add    circular 4.31456 ns/add speedup x3.95234    same statement: yes bytes 816/816
N=500   shift 60.968 ns/add circular 4.4123 ns/add  speedup x13.8177    same statement: yes bytes 8016/8016
N=5000  shift 1739.61 ns/add    circular 4.40188 ns/add speedup x395.198    same statement: yes bytes 80016/80016
```

N	Original (shift) ns/add	Redesign (circular) ns/add	Speedup	Same statement?	sizeof (original / redesign)
10	10.1	4.27	2.4x	yes	176 / 176
50	17.1	4.31	4.0x	yes	816 / 816
500	61.0	4.41	13.8x	yes	8016 / 8016
5000	1739.6	4.40	395x	yes	80016 / 80016

F. Defend the Final Design (2 marks)

- **Objective achieved:** (1) circular ns/add is 4.27 at N = 10 and 4.31 at N = 50 (1% change; within 20%); it stays 4.3-4.4 ns up to N = 5000. (2) Speedup at N = 50 is 4.0x (target 3x). (3) All four checksums are identical, so the statements are identical. (4) sizeof is the same. **Prediction check:** N = 50 speedup of at least 3x - correct (3.95x); N = 5000 more than 100x - correct (395x).
- **Where the prediction was only approximately right:** I predicted roughly linear growth for shifting, but 10 to 500 (50x more elements) cost only 6x more time. The compiler turns the shifting loop into a fast block copy (memmove with vector instructions), and up to about 8 KB the data sits in the L1 cache. At N = 5000 the 80 KB array no longer fits in L1 and the cost jumps to 1.7 microseconds. Big-O describes growth but hides such constant factors and cache effects.
- **New costs:** modulo arithmetic and wrap-around printing, and a risk of off-by-one bugs; the transactions are not contiguous in memory.
- **Where the original may still be preferable:** for the real requirement (10 or even 50 transactions per account) a shift costs 10-17 ns, and an ATM mini-statement happens a few times a day, so the absolute gain (6-13 ns per transaction) is negligible next to the cost of a database write or network call. The simple shifting array is easier to read, review and debug, keeps the history in order in memory, and is trivially correct. The redesign pays off when N is large or when adds are extremely frequent (batch posting of millions of transactions).
- **Conclusion:** the redesign meets its objective and is the better engineering choice if N is expected to grow; for N = 10 the original is acceptable and simpler. A std::deque would give the same O(1) but with heap allocation per block, which the fixed ring avoids.

Appendix - README.md and Git commands

README.md (copy into the repository)

README.md

```
# DSA01-<RegisterNumber>-Assignment2

Name: Logesh
Register Number: <RegisterNumber>
Course Code: DSA 0113
Compiler: g++ 13.3.0 (Ubuntu)
Compile command: g++ -std=c++17 -Wall -Wextra file.cpp -o file
Timing builds:   g++ -std=c++17 -O0|-O2 -DINLINE=inline|"__attribute__((noinline))" q6_bench.cpp

Folders: Q1 ... Q7 (source files, outputs, screenshots) Run:   cd Q1 && g++ -std=c++17 -Wall -Wextra

q1_three_ways.cpp -o q1 && ./q1   (same pattern for every file)

Results:
Q1  value leaves 45000; reference and pointer give 40500; Predict output: 1 12 13 / 1 100 / 16 13 6
Q2  highest() returns int&; dangling local reference warned and crashed (UB); vector growth invalidates references
Q3  original is ambiguous; redesigned int / double / Parcel overloads; charge(2)->int, charge(2.0f)->double
Q4  last-10 mini-statement with shifting array; 12 transactions keep the last 10; balance 6800
Q5  Token prints 101 103 104 4; BusTicket Seat 1,2,3 then Waiting; Issued 3
Q6  205 false; inline has no effect at -O0, 2.6x (friend) to 5.2x (getters) faster at -O2
Q7  shifting array vs circular buffer: 4.0x faster at N=50, 395x at N=5000, identical statements
```

Git commands (progressive commits - at least one per task)

Terminal

```
git init && git branch -M main mkdir Q1 Q2 Q3 Q4 Q5 Q6 Q7            # copy each task's .cpp files and
output screenshots into its folder git add Q1 && git commit -m "Q1: call by value/reference/pointer,
predict output, complete the code" git add Q2 && git commit -m "Q2: return by reference, dangling
reference warning, vector invalidation" git add Q3 && git commit -m "Q3: ambiguity error and redesigned
overloads" git add Q4 && git commit -m "Q4: SavingsAccount class with array of 10 transactions" git add
Q5 && git commit -m "Q5: static members, array of BusTicket, Token prediction" git add Q6 && git commit
-m "Q6: friend and inline functions, timings at -O0 and -O2" git add Q7 && git commit -m "Q7: diagnose,
circular buffer redesign, validation" git add README.md && git commit -m "Add README" git remote add
origin https://github.com/<username>/DSA01-<RegisterNumber>-Assignment2.git git push -u origin main
```